

令旗桌面客户端 Agent 技术文档

1. 技术概览

项目是一个 Windows 优先的 Electron 桌面应用，前端采用 React 18，使用 TypeScript 开发，Vite 负责构建。

层次	技术和职责
桌面容器	Electron 32, 创建窗口、文件选择、主进程能力
渲染进程	React 18 + TypeScript, 聊天、会话、设置、工具、MCP、Skill 界面
构建工具	Vite 5、 vite-plugin-electron
IPC 安全边界	contextBridge + 白名单 API, 渲染进程不直接访问 Node.js
模型协议	OpenAI 兼容 Chat Completions、Anthropic Messages
流式传输	SSE
MCP	@modelcontextprotocol/sdk , 本地 stdio transport
文件解析	pdf-parse 、 mammoth 、 xlsx 、 jszip 、 fast-xml-parser

主要目录：

```
src/ React █████
components/ Chat■Message■Sidebar■Settings █████
hooks/ ██████████
electron/
main.ts █████IPC■Agent Loop ██
preload.ts contextBridge ■■■ API
protocol.ts OpenAI/Anthropic ■■■ SSE ██
agentLoop.ts ████████████████
attachments.ts ██████████
mcp.ts MCP Server ██████████
skills.ts Skill ██████████
tools/ ████████████████████
shared/ ████████
tests/ ███████████████Skill ██
```

2. 运行时架构

```
React █████
Chat / Message / Sidebar / Settings / ToolList / SkillManager / McpManager
■
■ contextBridge + IPC
▼
Electron Preload
■ ■■■ API
▼
Electron █████
█████ . █████ . SSE ■■ . Agent Loop
█████ . ■■ . █████ . MCP ■■
■
```

```

■■■ ■■■■■■■■■■
■■■ Electron userData / localStorage
■■■ ■■ MCP Server
■■■ OpenAI/Anthropic ■■ API

```

渲染进程通过 `window.electronAPI` 调用有限的 IPC

方法，例如发送消息、选择目录、导入附件、查询工具、审批工具调用和管理 MCP。contextIsolation 开启，nodeIntegration 关闭，避免渲染页面直接取得 Node.js 能力。

3. Agent Loop

一次请求的主要流程如下:

1. 主进程接收消息、项目目录、权限模式、附件和已启用 Skill；
2. 预处理附件并发送任务状态；
3. 组合系统提示词，包括项目目录约束、权限模式和 Skill；
4. 将可执行工具的 schema 发送给模型；
5. 通过 SSE 增量接收模型文本和工具调用；
6. 执行工具并把结果追加回消息列表；
7. 如果工具是写入类且权限模式为 **ask**，创建审批请求并等待用户决定；
8. 最多循环 8 个工具步骤，最后一步不再提供工具，要求模型输出最终回答；
9. 返回 **completed**、**failed** 或 **cancelled** 状态。

任务状态包括：

```
queued, preparing_attachments, parsing_attachments, generating, waiting_approval, running_tool, completed, failed, cancelled.
```

4. 模型协议

OpenAI 兼容协议

请求地址为:

```
${baseUrl}/chat/completions
```

请求使用 Bearer Token，启用 **stream: true**。文本、工具调用和图片分别转换为 OpenAI 兼容消息格式。工具 schema 转换为 **type: function**。

Anthropic Messages 协议

请求地址为:

```
${baseUrl}/v1/messages
```

请求使用 `x-api-key` 和 `anthropic-version: 2023-06-01`。工具 schema 转换为 `input_schema`，工具结果转换为 `tool_result`，图片转换为 base64 内容块。

SSE 解析

主进程读取响应 body 的流，解析 `data:` 行，识别 `[DONE]`，并忽略无法解析的异常帧。用户停止任务时通过 `AbortController` 取消请求和工具执行。

5. 工具系统

工具由 `ToolRegistry` 注册，元数据包括名称、描述、权限、来源、分类和参数 schema。当前内置工具：

工具	权限	作用
<code>time_current</code>	read	获取当前时间
<code>calculator_calculate</code>	read	计算四则运算、取模和括号表达式
<code>file_list</code>	read	列出项目目录内容，最多返回 200 项
<code>file_read</code>	read	读取项目内文本文件，最多返回 100,000 字符
<code>file_write</code>	write	以 UTF-8 写入文件，最多 200,000 字符
<code>http_fetch</code>	read	发送 HTTP/HTTPS GET，响应最多保留 20,000 字符

工具执行器统一负责 JSON 参数解析、schema 校验、30 秒超时、取消、结果截断和错误封装。危险权限工具当前不会暴露给模型。

路径安全

读取路径必须解析到项目目录内的真实路径。写入路径在创建新目录时也会检查真实父目录，拒绝 `../` 越界和通过符号链接逃逸项目根目录。完全访问模式只放宽写入目标边界，不会改变参数校验和工具超时。

6. 审批机制

写入工具在 `ask` 模式下会创建审批请求，审批请求包含：

- 请求 ID、会话 ID、工具调用 ID；
- 工具名称和原始参数；
- 参数 SHA-256 哈希；
- 目标路径、覆盖状态、内容长度和预览；
- 创建时间、过期时间和当前状态。

用户批准时必须提交与创建时一致的参数哈希，防止审批后参数被篡改。审批结果为批准、拒绝、过期或取消。待处理审批会保存到 Electron `userData/pending-approvals.json`。

7. 附件处理

附件先复制到：

```
app.getPath('temp')/attachments/<■■■■>/
```

导入时按扩展名和文件头双重校验，避免仅修改扩展名绕过检查。文本、PDF、DOCX、XLSX/XLS、PPTX 在主进程只读解析；图片读取为 data URL。Office 文件不执行宏、脚本、外部链接或公式计算。

常量限制：

- `MAX_ATTACHMENT_SIZE` = 20 1024 1024
- `MAX_ATTACHMENTS` = 5
- `MAX_EXTRACTED_CHARS` = 100000
- `MAX_TOTAL_EXTRACTED_CHARS` = 180000
- Office ZIP 条目不超过 2000 个，解压总大小不超过 50 MB

任务结束后附件延迟 10 分钟清理；应用启动时清理超过 1 小时的附件目录。

8. 会话、配置和数据位置

渲染进程 localStorage

- `api-config`：API 密钥、地址、模型和协议；
- `chat-conversations-state`：会话、项目、消息、权限模式和会话启用的 Skill ID。

Electron userData

- `pending-approvals.json`：审批状态；
- `mcp-servers.json`：MCP Server 配置；
- `skills/`：全局 Skill 目录。

实际 `userData` 路径由 Electron `app.getPath('userData')` 决定。发布环境不应假定它固定在项目目录中。

9. MCP 与 Skill

MCP 通过 `StdioClientTransport` 启动本机进程，调用 `listTools` 后把工具动态注册到统一工具注册表。工具名称使用 `mcp_■■■■_■■■■` 命名空间。`readOnlyHint` 映射为 `read`，`destructiveHint` 映射为 `dangerous`，未标注的工具按 `write` 处理。

Skill 扫描全局 `userData/skills` 和项目 `.agent/skills` 的一级子目录，每个目录必须有 `SKILL.md`。文件需包含 frontmatter 和非空正文，单个文件最大 64 KB，单次注入正文最大 200 KB。项目同名 Skill

覆盖全局 Skill。Skill 只增加系统提示词，不执行脚本，也不能绕过审批。

10. 构建和验证

```
npm ci
npm run dev
npm run build
npm test
npm run dist:win
```

`npm run build` 会执行 TypeScript 检查并构建 `dist/`、`dist-electron/`。`npm test` 覆盖附件限制、审批持久化、路径安全和 Skill 解析。Windows 安装包和免安装目录输出到 `release/`。

11. 当前边界

- 只支持本地 `studio` MCP，不支持远程 HTTP MCP；
- `http_fetch` 仅支持 GET，不支持 POST、PUT、PATCH；
- 当前没有服务端账号体系、共享知识库或向量数据库；
- 会话和 API 配置以本地存储为主，不是多端同步数据；
- 图片理解依赖所配置模型的多模态能力，客户端不内置 OCR；
- Electron 发布包未包含 OpenViking 等独立知识库服务。